

GLYPH-X

a super regular RISC architecture

June 29, 2025 – v0.6.0-current

Michael Clark

Independent Researcher



Maya glyphs in stucco on display at the Museo de Sitio in Palenque, Mexico.

Image courtesy of Wikipedia. Public domain.

to my mentors, whose guidance and wisdom made this work possible.

*"Perfection is achieved,
not when there is nothing more to add,
but when there is nothing left to take away."*

— Antoine de Saint-Exupéry

Contents

Preface	v
1. Architecture	1
1.1. Introduction	1
1.1.1. load-store	2
1.1.2. registers	2
1.1.3. memory	2
1.1.4. instructions	3
1.1.5. constants	4
1.2. Core concepts	5
1.3. Program streams	6
1.4. Instruction format	7
1.4.1. instruction templates	7
1.4.2. instruction sizes	8
1.4.3. instruction forms	8
1.4.4. instruction decoding	9
1.5. Register file	10
1.6. Example pipeline	11
2. System	12
2.1. User-level registers	12
2.1.1. floating-point math registers	13
2.1.2. clock and frequency registers	14
2.1.3. address and domain registers	14
2.2. System registers	15
2.2.1. privileged trap registers	16
2.2.2. privileged system registers	18
2.2.3. privileged debug registers	20
2.3. Capability extension	21
2.3.1. color permission registers	22
2.3.2. color capability registers	23
2.3.3. color matrix registers	24
2.4. Domain extension	25
2.4.1. domain status registers	26
2.4.2. domain routing registers	26
2.4.3. domain context registers	27

2.5. Address translation	28
2.5.1. page table structure	28
2.5.2. page table entries	28
2.5.3. page table addresses	29
2.5.4. page table translation	29
3. Instructions	31
3.1. Instruction listing — 16-bit	31
3.1.1. break	31
3.1.2. j	31
3.1.3. b	31
3.1.4. ibj	32
3.1.5. link	32
3.1.6. movh	33
3.1.7. movw	33
3.1.8. movi	33
3.1.9. addi	33
3.1.10. srli	34
3.1.11. srai	34
3.1.12. slli	34
3.1.13. addh	34
3.1.14. leapc	35
3.1.15. loadpc	35
3.1.16. storepc	35
3.1.17. load	35
3.1.18. store	36
3.1.19. compare	36
3.1.20. logic	37
3.1.21. pin	37
3.1.22. and	38
3.1.23. or	38
3.1.24. xor	38
3.1.25. add	38
3.1.26. srl	39
3.1.27. sra	39
3.1.28. sll	39
3.1.29. sub	39
3.1.30. mul	40
3.1.31. div	40
3.1.32. illegal	40
4. Assembly	41
4.1. Introduction	41

Contents

4.2. Concepts	41
4.2.1. assembly	41
4.2.2. object	41
4.2.3. archive	41
4.2.4. executable	41
4.2.5. shared library	42
4.2.6. section	42
4.2.7. segment	42
4.2.8. symbol	42
4.2.9. relocation	42
4.2.10. program linking	42
4.3. Directives	43
4.4. Pseudo-instructions	44
4.5. Calling convention	45
4.5.1. calling convention — 16-bit	45
A. Appendix	46
A.1. Opcode summary — 16-bit	46
References	47

Preface

this document introduces a RISC architecture designed with simplicity as its dominating principle, along with several features not yet present in mainstream RISC architectures:

- a compression technique for split instruction and constant streams.
- a variable-length instruction format designed for vectorized decoding.
- an efficient frontier between hardware synthesis and software translation.

the document begins with a general introduction to RISC architectures, including principal elements such as load-store operation, memory, registers, instructions, and constants. it then presents an overview of the proposed architecture, followed by sections on program streams, the variable-length instruction format, the instruction set, its assembly syntax, and the programming language calling convention.

this document does not simply present an architecture that is a reimplementa-tion of existing ideas; rather, it introduces a novel architecture with an instruction coding scheme optimized for efficient parallel decoding in hardware or software, combined with contemporary practice in instruction semantics.

this work is intended for computer architects, systems programmers, students of computing machinery, and anyone interested in the evolution of RISC design. by the end of the document, readers should be able to reason about computer programs at the instruction level and understand the logic behind the architecture’s design choices—with enough detail to implement the architecture in hardware or software.

1. Architecture

1.1. Introduction

this section gives a brief introduction to RISC architectures.

A RISC¹ machine is a type of general-purpose computer with the characteristic that it has a reduced set of instructions in contrast to a CISC² machine. A RISC machine is Turing complete meaning it can perform any computation that a Turing machine can, given enough time and memory. a Turing machine [11] is a theoretical model of computation.

a RISC machine has a set of instructions which comprise basic operations such as: *load-from-memory*, *store-to-memory*, *add*, *subtract*, *compare*, plus *conditional branch* and *unconditional branch* instructions et cetera; which one can imagine as a list of instructions on a paper tape. each instruction has an *opcode*, which is a unique binary pattern that identifies the *operation*, plus several *operands*, which are arguments to the instruction.

```
register-0 = load-from-memory at tape-address-0
register-1 = load-from-memory at tape-address-1
register-2 = add register-0 and register-1
            store-to-memory register-2 at tape-address-2
```

some instructions have operands that point to values inside of *registers* in a *register-file* which is like a close filing cabinet containing cards with numbers on them, and some of these numbers are addresses that point to values in *main-memory* which is like a larger but slower filing cabinet. some of these values are *immediate* values which are small numbers listed inside of the instructions on the paper tape.

the paper tape is just a way conceptualize a list of instructions stored in *main-memory*. there is a special register called *PC* short for *program counter*, which points to the current position on the tape. after each instruction executes the tape is advanced to the next instruction and the *program counter* is incremented, until it encounters a *branch* instruction which causes it to move forwards or backwards to a different position on the tape. branch instructions can be *conditional* or *unconditional*. conditional branches are selectively executed based on the results of a comparison instruction.

¹Reduced Instruction Set Computer

²Complex Instruction Set Computer

1. Architecture

1.1.1. load-store

a *load-store* architecture [3] is a way to characterize RISC architectures where most instructions have simple operands that point to values held in registers, plus *load* and *store* instructions to retrieve and commit values to main memory. a *load-store* architecture alleviates the need to add complex addressing modes, plus input-output to peripherals and secondary storage use *MMIO*³ to avoid needing special *Input/Output* instructions.

1.1.2. registers

registers are temporary storage used to fill input and output operands for the *ALU*⁴ before and after execution of instructions. registers are organized as a word-addressable store where each register number refers to $XLEN \in \{64, 128\}$ bits of data. *XLEN* is a parameter that specifies the width of registers in bits.

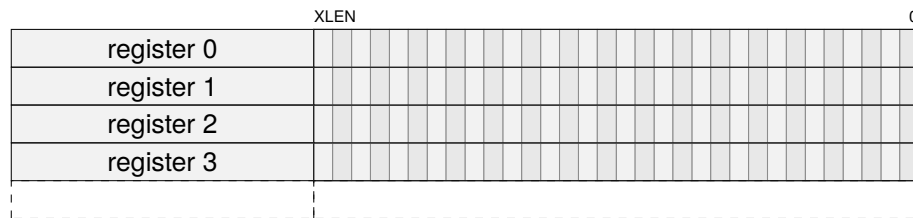


Figure 1.1.: organization of register storage.

1.1.3. memory

main memory is primary storage which in modern computers is most likely *DRAM*⁵. main memory is organized as a byte-addressable store where each address refers to a byte which is 8-bits of data. *ALEN* is a parameter that specifies the width of addresses in bits.

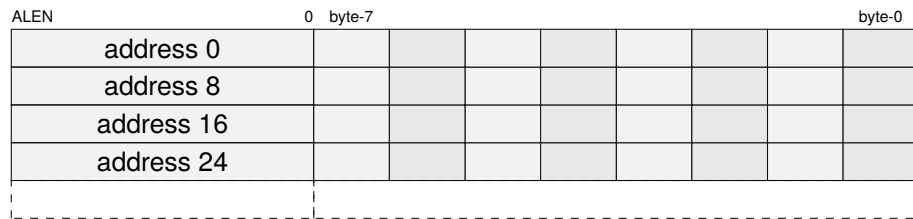


Figure 1.2.: organization of main-memory storage.

³*MMIO* - Memory-mapped I/O.

⁴*ALU* - Arithmetic logic unit.

⁵*DRAM* - Dynamic random-access memory.

1. Architecture

1.1.4. instructions

instruction memory is a memory region dedicated to program instructions. it is organized as a word-addressable store, similar to main memory, but is read-only and reserved exclusively for instructions. this configuration is typically referred to as a Harvard architecture [1], in contrast to a Von Neumann architecture [8], which uses a shared memory region for both program instructions and data.

instruction words contain packets of 16-bits which are composed of size, opcode, and operand fields. instruction packets can be appended together to form larger instruction words with larger opcode and operand fields. section 1.4 provides details on the instruction template and instruction forms.

15	7	6	2	1	0
<i>operand</i> _[8:0]			<i>opcode</i> _[4:0]		<i>sz</i> _[1:0]

Figure 1.3.: 16-bit instruction containing one packet.

15	7	6	2	1	0
<i>operand</i> _[8:0]			<i>opcode</i> _[4:0]		<i>sz</i> _[1:0]
<i>operand</i> _[17:9]			<i>opcode</i> _[9:5]		<i>sz</i> _[3:2]

Figure 1.4.: 32-bit instruction containing two packets.

instruction words are packed together into instruction blocks containing instruction size information that is used to fuse them together to compose variable-length instructions [7]. instruction words can begin on any 16-bit aligned address. instruction memory is addressed with displacements from the *program counter* called *pc-relative* addresses.

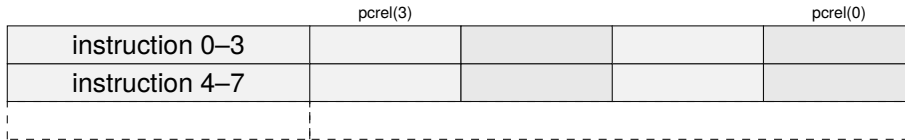


Figure 1.5.: organization of instruction-memory with 16-bit instructions.

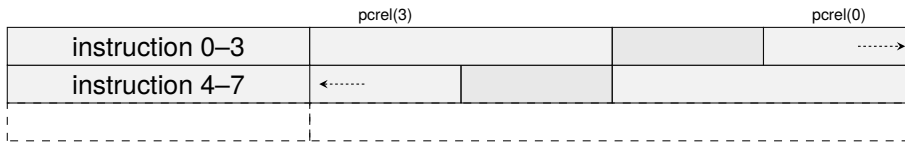


Figure 1.6.: organization of instruction-memory with 16-bit and 32-bit instructions.

1. Architecture

1.1.5. constants

constant memory [10] is a memory region dedicated to constants. it is organized as a word-addressable store like main-memory, but is read-only and restricted to constants. instructions can refer to constants using *ib-relative* addresses inside instruction operands. section 1.3 provides more details on the split instruction and constant streams.

constant memory is addressed with displacements from the *immediate base* register called *ib-relative* addresses which are stored inside instruction operand slots, are sized, scaled and aligned addresses computed relative to the *immediate base* register, which is like a program counter for constants. these diagrams show $ib8(n)$ through $ib64(n)$ *ib-relative* addresses for access to 8-bit through to 64-bit constants respectively: *note: ib-relative addresses alias a single constant storage space containing all types.*

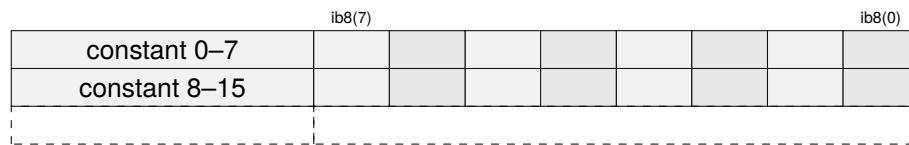


Figure 1.7.: organization of *ib8* constant-memory storage.

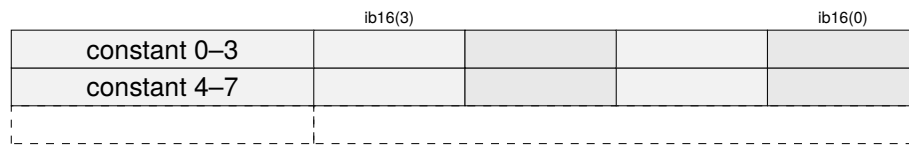


Figure 1.8.: organization of *ib16* constant-memory storage.

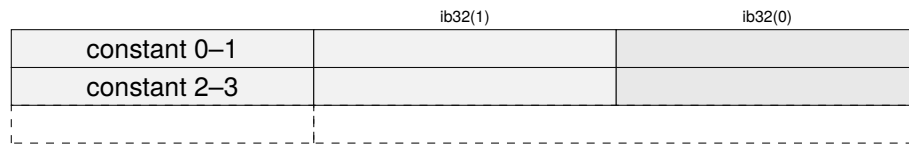


Figure 1.9.: organization of *ib32* constant-memory storage.

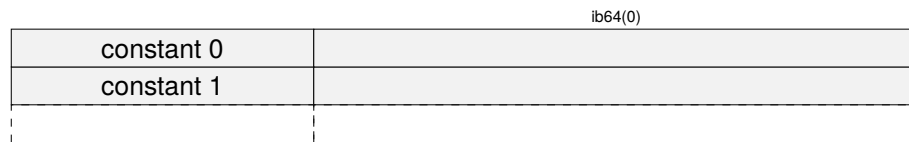


Figure 1.10.: organization of *ib64* constant-memory storage.

1.2. Core concepts

glyph is a super regular RISC architecture that encodes constants in a secondary stream accessed via an *immediate base* register that points at immediate blocks containing constants accessed via a constant address mode. the *immediate base* register branches like the *program counter*, and procedure calls and returns set and restore (*pc,ib*) together.

glyph uses relative address vectors in its link register which is different to typical RISC architectures. glyph does this so that the branch instructions can fit (*pc,ib*) into the a single link register for compatibility with traditional RISC architectures. glyph achieves this by packing two relative (*pc,ib*) displacements into a relative address vector⁶.

immediate blocks can be switched using the immediate block branch instruction. immediate blocks, unlike typical RISC architectures, mean that most relocations are word sized like CISC architectures, and can use C-style structure packing and alignment rules.

this list outlines some differentiating elements of the super regular RISC architecture:

- variable length instruction format supporting 16, 32, 64, and 128-bit instructions.
- 16-bit compressed instruction packets can access 8 registers.
- (*pc,ib*) is a *program counter* and *immediate base* register address vector.
- link register contains a packed relative (*pc,ib*) address vector to function entry.
- *ibj immediate-block-jump* adds a relative address to the *immediate base* register.
- *movw move-word-immediate-block* uses an unsigned displacement to access constants.
- *jalib jump-and-link-immediate-block* or *call link function* links address vector and adds constants to (*pc,ib*) and is used to branch the *program counter* and *immediate base* register at the same time for calling procedures.
- *jtlib jump-to-link-immediate-block* or *return link function* subtracts link vector from and adds constants to (*pc,ib*) and is used to branch the *program counter* and *immediate base* register at the same time for returning from called procedures.
- *pin pack-indirect* packs two absolute addresses as relative address vector from (*pc,ib*) and is used for calling absolute addresses such as virtual functions.

the *pin* and *link* instructions form a modular arithmetic ring of relative address vectors, due to their use of relative addresses. relative address vectors can be encrypted, decrypted, and authenticated to form a verifiable chain of addresses for control flow integrity.

⁶the architecture defines two parameters: *ALEN* and *XLEN*, which respectively to refer to width of addresses and width of general purpose registers in bits. when $XLEN \geq ALEN \times 2$ it is possible to pack absolute addresses instead of relative addresses, as would be the case where $ALEN=64$ and $XLEN=128$.

1.3. Program streams

glyph separates the stored programs into two streams, one with instructions and one with constants [10]. the instruction stream is addressed with the *program counter* (*pc*) and the constant stream is addressed with the *immediate base* (*ib*) register. this could be referred to as split program stream Harvard architecture.

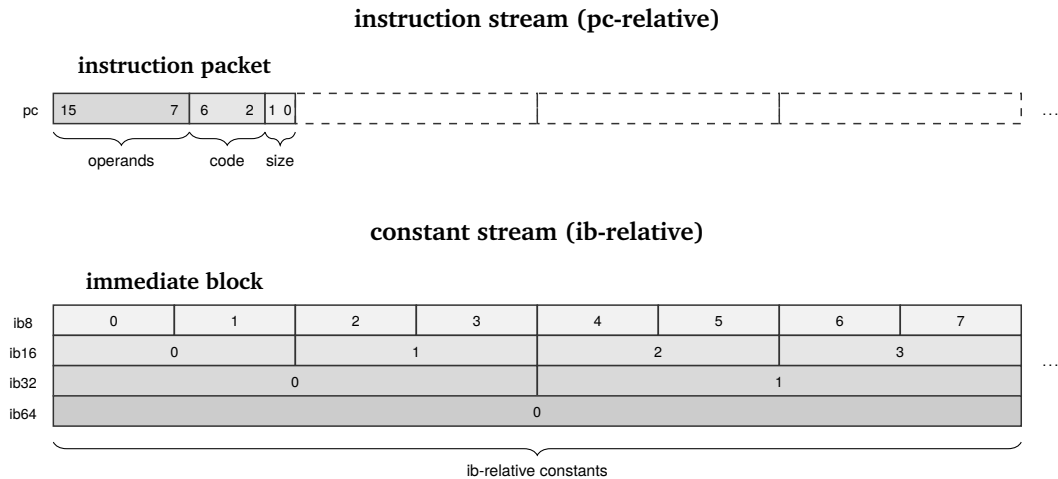


Figure 1.11.: program counter and immediate base register.

instruction blocks are aligned memory blocks addressed by the *program counter*. instruction memory is addressed with *pc-relative* addresses in instruction immediate values or indirectly with constants accessed using *ib-relative* addresses stored inside instruction operand slots.

immediate blocks are aligned memory blocks addressed by the *immediate base* register. constant memory is addressed with *ib-relative* addresses stored inside instruction operand slots, which are sized, scaled, and aligned relative to the *immediate base* register.

the constant stream branches independently using the *immediate-block-jump* instruction to update the *immediate base* register, or together with the *program counter* in procedure call and return instructions that branch instruction and constants at the same time; *jump-and-link-immediate-block* and *jump-to-link-immediate-block* add and subtract address vectors to (*pc,ib*). the *pack-indirect* instruction allows absolute (*pc,ib*) addresses to be packed into an address vector for indirect calls.

the instruction forms use bonded operand slots for immediate operands and operand bits do not overlap opcode bits. the use of immediate blocks means large immediate constants can all be accessed using short references encoded inside of operand slots for instructions that use an immediate block relative addressing mode.

1.4. Instruction format

instructions use a variable length format [7] with a 16-bit instruction packet that supports 16, 32, 64, and 128-bit instruction words. the instruction packet uses a *super regular* scheme, whereby successive instruction packets extend the fields in the previous instruction packet [6]. instruction words can begin on any 16-bit aligned address.

1.4.1. instruction templates

the variable length instruction format has a single base format where fields in the template instruction form are extended by successive instruction packets.

15	7	6	2	1	0
<i>operand</i> _[8:0]		<i>opcode</i> _[4:0]		<i>sZ</i> _[1:0]	

Figure 1.12.: instruction template — 16-bit.

15	7	6	2	1	0
<i>operand</i> _[8:0]		<i>opcode</i> _[4:0]		<i>sZ</i> _[1:0]	
<i>operand</i> _[17:9]		<i>opcode</i> _[9:5]		<i>sZ</i> _[3:2]	

Figure 1.13.: instruction template — 32-bit.

15	7	6	2	1	0
<i>operand</i> _[8:0]		<i>opcode</i> _[4:0]		<i>sZ</i> _[1:0]	
<i>operand</i> _[17:9]		<i>opcode</i> _[9:5]		<i>sZ</i> _[3:2]	
<i>operand</i> _[26:18]		<i>opcode</i> _[14:10]		<i>sZ</i> _[5:4]	
<i>operand</i> _[35:27]		<i>opcode</i> _[19:15]		<i>sZ</i> _[7:6]	

Figure 1.14.: instruction template — 64-bit.

15	7	6	2	1	0
<i>operand</i> _[8:0]		<i>opcode</i> _[4:0]		<i>sZ</i> _[1:0]	
<i>operand</i> _[17:9]		<i>opcode</i> _[9:5]		<i>sZ</i> _[3:2]	
<i>operand</i> _[26:18]		<i>opcode</i> _[14:10]		<i>sZ</i> _[5:4]	
<i>operand</i> _[35:27]		<i>opcode</i> _[19:15]		<i>sZ</i> _[7:6]	
<i>operand</i> _[44:36]		<i>opcode</i> _[24:20]		<i>sZ</i> _[9:8]	
<i>operand</i> _[53:45]		<i>opcode</i> _[29:25]		<i>sZ</i> _[11:10]	
<i>operand</i> _[62:54]		<i>opcode</i> _[34:30]		<i>sZ</i> _[13:12]	
<i>operand</i> _[71:63]		<i>opcode</i> _[39:35]		<i>sZ</i> _[15:14]	

Figure 1.15.: instruction template — 128-bit.

1. Architecture

1.4.2. instruction sizes

the variable length instruction format has a 2-bit size field inspired by LEB128 [4] in a fixed position in every instruction packet, to reduce the complexity of size-decoding for variable length instructions⁷. this table shows the size fields for the different instruction sizes:

Instruction size	Size vector
16-bit	{00}
32-bit	{01, 11}
64-bit	{10, 11, 11, 11}
128-bit	{11, 11, 11, 11, 11, 11, 11, 11}

Table 1.1.: variable-length instruction size fields

1.4.3. instruction forms

the instructions forms are super regular in that operand and opcode bits do not overlap and the number and complexity of the formats is reduced so that vectorized instruction decoding is simple in both hardware and software. the scheme is designed so that 1-bit of coding space in the larger packet can be used to extend register sizes.

16-bit instruction forms

this section details the layout of the 16-bit instruction forms:

15	7	6	2	1	0
$imm_{[5:0]}$				$opcode_{[4:0]}$	
				00	

Figure 1.16.: 16-bit large immediate.

15	13	12	7	6	2	1	0
$rc_{[2:0]}$		$imm_{[5:0]}$			$opcode_{[4:0]}$		00

Figure 1.17.: 16-bit one operand with immediate.

15	13	12	10	9	7	6	2	1	0
$rc_{[2:0]}$		$rb_{[2:0]}$		$imm_{[2:0]}$		$opcode_{[4:0]}$		00	

Figure 1.18.: 16-bit two operand with immediate.

15	13	12	10	9	7	6	2	1	0
$rc_{[2:0]}$		$rb_{[2:0]}$		$ra_{[2:0]}$		$opcode_{[4:0]}$		00	

Figure 1.19.: 16-bit three operand.

⁷instructions are considered *well-formed* if the size field of subsequent packets has the value 11.

1. Architecture

32-bit instruction forms

this section details the layout of the 32-bit instruction forms:

15	7	6	2	1	0
<i>imm</i> _[8:0]		<i>opcode</i> _[4:0]		01	
<i>imm</i> _[17:9]		<i>opcode</i> _[9:5]		11	

Figure 1.20.: 32-bit large immediate.

15	13	12	7	6	2	1	0
<i>rc</i> _[2:0]		<i>imm</i> _[5:0]		<i>opcode</i> _[4:0]		01	
<i>rc</i> _[5:3]		<i>imm</i> _[11:6]		<i>opcode</i> _[9:5]		11	

Figure 1.21.: 32-bit one operand with immediate.

15	13	12	10	9	7	6	2	1	0
<i>rc</i> _[2:0]		<i>rb</i> _[2:0]		<i>imm</i> _[2:0]		<i>opcode</i> _[4:0]		01	
<i>rc</i> _[5:3]		<i>rb</i> _[5:3]		<i>imm</i> _[5:3]		<i>opcode</i> _[9:5]		11	

Figure 1.22.: 32-bit two operand with immediate.

15	13	12	10	9	7	6	2	1	0
<i>rc</i> _[2:0]		<i>rb</i> _[2:0]		<i>ra</i> _[2:0]		<i>opcode</i> _[4:0]		01	
<i>rc</i> _[5:3]		<i>rb</i> _[5:3]		<i>ra</i> _[5:3]		<i>opcode</i> _[9:5]		11	

Figure 1.23.: 32-bit three operand.

1.4.4. instruction decoding

the instruction size encoding and field extension scheme has been designed to minimize complexity for parallel decoding in hardware and software. the following table shows the instruction width combinations for various width parallel instruction decoders.

Packets	Bits	Bytes	n_{combos}	$\log_2(n)$
1-wide	16-bit	2	1	0
2-wide	32-bit	4	16	4
4-wide	64-bit	8	58	5.86
8-wide	128-bit	16	574	9.16
16-wide	256-bit	32	51904	15.66

Table 1.2.: instruction decode combinations

1.5. Register file

the glyph register file is extensible due to the variable length instruction format and supports a different number of registers depending on the instruction size.

- 16-bit instruction packet can access 8 registers with up to 3 operands.
- 32-bit instruction packet can access 64 registers with up to 3 operands.
- 64-bit instruction packet can access 64 registers with up to 6 operands.

the register state accessible by the 16-bit instruction packet is comprised of:

- *program counter* register (aligned to 2 bytes).
- *immediate base* register (aligned to 64 bytes).
- 8 × general purpose registers (*r0* through *r7*).
- 1 × predicate register (*flag*).

the following diagram shows the register state accessible by the 16-bit instruction packet:

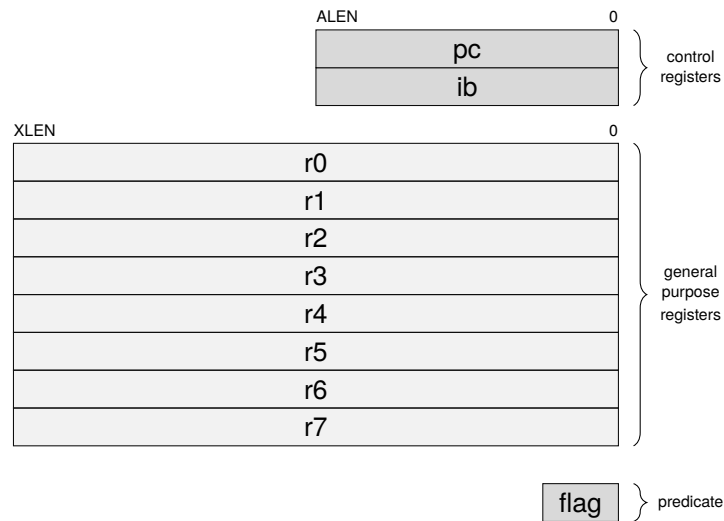


Figure 1.24.: register state accessible by 16-bit instruction packet.

the use of $ALEN^8$ and $XLEN^9$ parameters is to indicate that the width of addresses can be less than the width of the general purpose registers.

⁸ $ALEN$ refers to the width of addresses in bits.

⁹ $XLEN$ refers to the width of general purpose registers in bits.

1.6. Example pipeline

an illustrative micro-architecture is proposed based on the classic 5-stage RISC micro-architecture [9] with the addition of an *operand fetch* stage and a *constant memory* port. this revised 6-stage micro-architecture is composed of the following pipeline stages:

- IF — *instruction fetch*: reads instructions from memory into a fetch buffer.
- ID — *instruction decode*: decodes instruction length, opcode, and operands.
- OF — *operand fetch*: reads operands from register file and constant memory.
- EX — *execute*: performs logical operations or arithmetic on the operands.
- MA — *memory access*: loads data from or stores data to memory.
- WB — *writeback*: writes results back to the register file.

a simplified micro-architecture using those pipeline stages might look like this: this example omits hazard detection and forwarding logic for the sake of simplicity.

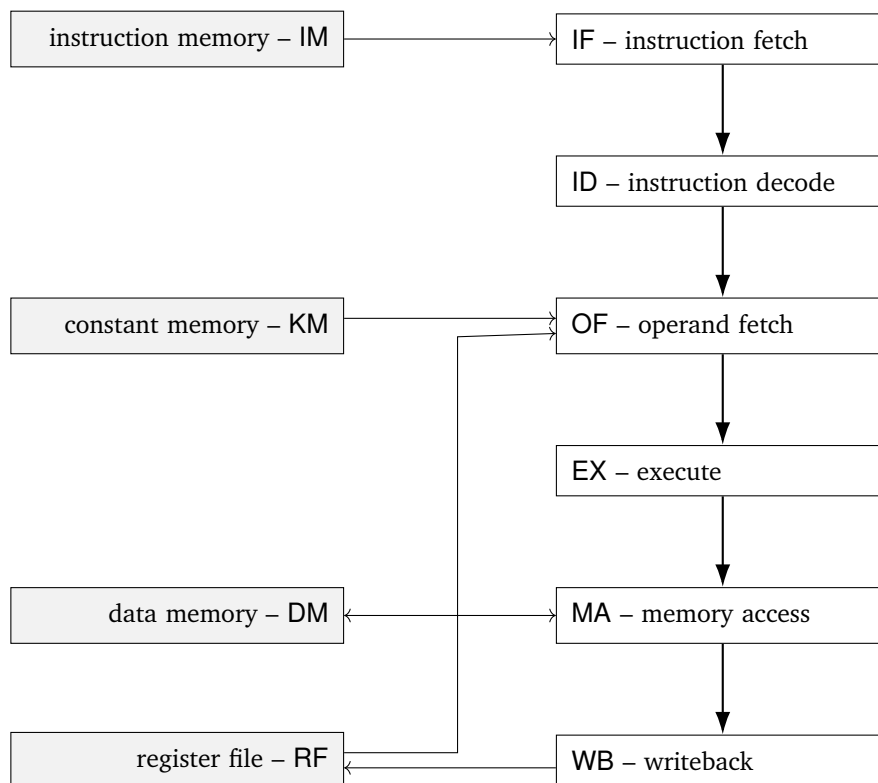


Figure 1.25.: sample 6-stage micro-architecture with support for constant memory.

2. System

2.1. User-level registers

the architecture provides several user-level status and control registers for floating-point status and control, clock time, clock frequency, thread address and thread domain.

user-level status and control registers can be read and written using the `sysread`, `syswrite`, `sysset`, and `sysclear` instructions. user-level status and control registers are available in contexts where their associated bit is enabled in `scontrol`.

user-level registers

the following table lists the user-level registers:

no.	name	description
floating-point math registers		
0x00	fpstatus	floating-point status
0x01	fpcontrol	floating-point control
clock and frequency registers		
0x02	ctime	clock time
0x03	cfreq	clock frequency
address and domain registers		
0x04	addr	user thread address
0x05	domain	user domain address

Table 2.1.: user-level registers

2. System

2.1.1. floating-point math registers

the section describes the user-level floating-point math registers.

floating-point status (fpstatus)

fpstatus is a read-only register containing status information for floating-point operations. the I, Z, U, O, and P fields contain accrued floating-point exceptions. this register is visible if syscontrol.F is enabled.

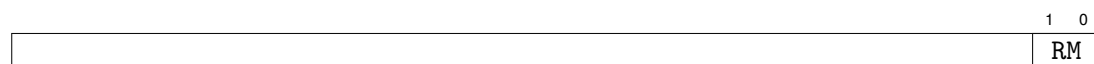


no.	name	description
1 << 0	I	invalid operation
1 << 1	Z	divide by zero
1 << 2	U	numeric underflow
1 << 3	O	numeric overflow
1 << 4	P	inexact result

Table 2.2.: floating-point status flags

floating-point control (fpcontrol)

`fpcontrol` is a read-write register containing control information for floating-point operations. the `RM` field contains the current floating-point rounding mode. this register is visible if `syscontrol.F` is enabled.



no.	name	description
0	RN	round to nearest (even)
1	RD	round down (towards $-\infty$)
2	RU	round up (towards $+\infty$)
3	RZ	round towards zero (truncate)

Table 2.3.: floating-point control round mode field

2. System

2.1.2. clock and frequency registers

the section describes the user-level clock and frequency registers.

clock time (ctime)

`ctime` is a read-only register containing the wall-clock tick counter since power on in clock tick units denoted by `cfreq`. this register is visible if `syscontrol.T` is enabled.

clock frequency (cfreq)

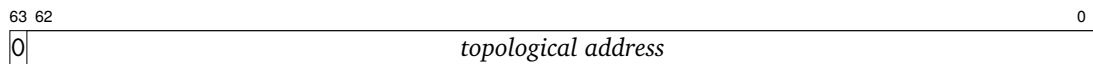
`cfreq` is a read-only register containing the wall-clock tick interval in picoseconds; $\frac{10^{12}}{f}$ where f is the frequency in Hertz (Hz). this register is visible if `syscontrol.T` is enabled.

2.1.3. address and domain registers

the section describes the user-level address and domain registers.

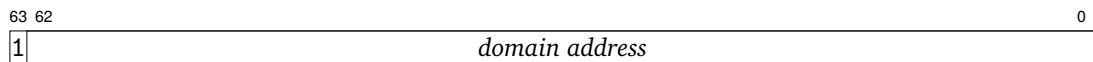
thread address (addr)

`addr` is a read-only register containing an alias of the unique system-wide topological address for this thread from `saddr`. this register is visible if `syscontrol.D` is enabled.



domain address (domain)

domain is a read-only register containing an alias of the system-wide domain address for this thread from sdomain. this register is visible if syscontrol.D is enabled.



2.2. System registers

the architecture provides several privileged status and control registers for trap handling, system configuration, address translation, timers, interrupts, and debugging.

privileged system status and control registers can be read and written using the `sysread`, `syswrite`, `sysset`, and `sysclear` instructions. privileged system registers are accessible from executable pages with a supervisor or physical address spaces.

privileged registers

the following table lists the privileged trap, system and debug registers:

no.	name	description
privileged trap registers		
0x10	tstatus	trap status
0x11	tcontrol	trap control
0x12	tscratch	trap scratch
0x13	tvalue	trap value
0x14	tcause	trap cause
0x15	tack	trap acknowledge
0x16	tepc	trap exception program counter
0x17	teib	trap exception immediate base
0x18	thpc	trap handler program counter
0x19	thib	trap handler immediate base
privileged system registers		
0x21	scontrol	system control
0x22	sfeature	system feature
0x23	sptr	system page table root
0x24	saddr	system thread address
0x25	sdomain	system domain address
0x26	starget	system target address
0x27	smessage	system message interrupt
0x28	stimer	system deadline timer
0x29	sie	system interrupt enable
0x2a	sip	system interrupt pending
privileged debug registers		
0x30	dstatus	debug status
0x32	dcycle	debug cycle counter
0x33	dinst	debug instruction counter
0x34	dstop	debug stop instruction
0x35	dfetch	debug monitor fetch address
0x36	dread	debug monitor read address
0x37	dwrite	debug monitor write address

Table 2.4.: privileged registers

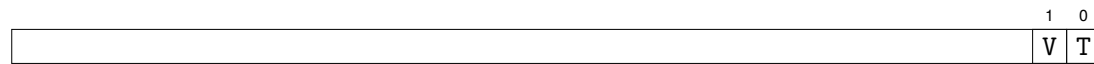
2. System

2.2.1. privileged trap registers

the section describes the privileged trap registers.

trap status (tstatus)

tstatus is a read-only register containing trap status. the T field indicates whether a timer interrupt is pending. the V field indicates whether a virtual interrupt is pending.

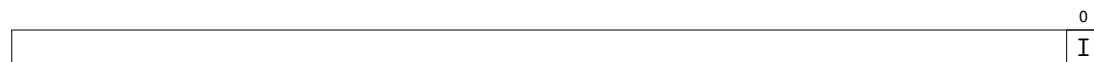


no.	name	description
1 << 0	T	timer interrupt pending
1 << 1	V	virtual interrupt pending

Table 2.5.: trap status fields

trap control (tcontrol)

tcontrol is a read-write register containing control bits. the I field is read-write and controls whether system interrupts are enabled for this hardware thread.



no.	name	description
1 << 0	I	system interrupt enable

Table 2.6.: trap control fields

trap scratch (tscratch)

tscratch is a read-write save register for use during trap handling.

trap value (tvalue)

tvalue is a read-only register containing the identity of the trap, and may contain:

- the faulting instruction word for *break* and *illegal instruction exceptions*.
- the faulting address for *access* and *page faults*.
- the message value for *virtual interrupts*.
- the system clock time for *timer interrupts*.
- the domain number for *domain faults* when the domain extension is enabled and *capability faults* when both capabilities and domain extensions are enabled.

2. System

trap cause (tcause)

tcause is a read-only register that contains the cause of the current trap.

no.	name	description
system exceptions		
1	break-instruction	break instruction exception
2	illegal-instruction	illegal instruction exception
3	debug-monitor	debug monitor exception
4	misaligned-fetch	fetch misaligned
5	misaligned-load	load misaligned
6	misaligned-store	store misaligned
7	access-fault-fetch	fetch access fault
8	access-fault-load	load access fault
9	access-fault-store	store access fault
10	page-fault-fetch	fetch page fault
11	page-fault-load	load page fault
12	page-fault-store	store page fault
13	capability-fault	capability fault
14	domain-fault	domain fault
system interrupts		
30	timer-interrupt	timer interrupt
31	virtual-interrupt	virtual interrupt
32 . . . 63	interrupt-n	interrupt pins 0 – 31

Table 2.7.: trap causes

trap acknowledge (tack)

tack is a write-only register where the trap cause is written to acknowledge the trap so that interrupts are not delivered while state is being saved, and for double-faults to be detected.

trap exception program counter (tepc)

tepc is a read-only register which contains the *program counter* address before the trap.

trap exception immediate base (teib)

teib is a read-only register which contains the *immediate base* address before the trap.

trap handler program counter (thpc)

thpc is a read-write register containing the address of the system trap handler routine.

trap handler immediate base (thib)

thib is a read-write register containing the address of the system trap handler constants.

2. System

2.2.2. privileged system registers

the section describes the privileged system registers.

system control (scontrol)

scontrol is a read-write register containing system control bits.



no.	name	description
1 << 0	F	floating-point math enabled
1 << 1	T	user clock and frequency enabled
1 << 2	D	user address and domain enabled
1 << 3	A	user address space access disable enabled
1 << 4	E	user address space execute disable enabled
1 << 5	P	physical permissions check enabled
1 << 6	C	capabilities check enabled
1 << 7	V	virtual domains enabled

Table 2.8.: system control fields

system feature (sfeature)

sfeature is a read-only register containing system feature bits.



no.	name	description
1 << 0	F	floating-point math feature
1 << 1	T	user clock and frequency feature
1 << 2	D	user address and domain feature
1 << 3	A	user address space access disable feature
1 << 4	E	user address space execute disable feature
1 << 5	P	physical permissions check feature
1 << 6	C	capabilities check feature
1 << 7	V	virtual domains feature

Table 2.9.: system feature fields

system page table root (sptr)

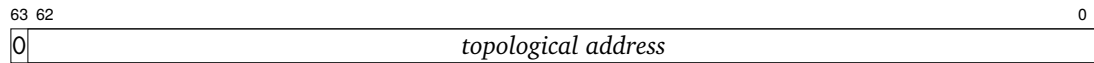
sptr is a read-write register containing the page table root for address translation on this hardware thread. the *ASID* field contains the address space identifier for this thread.



2. System

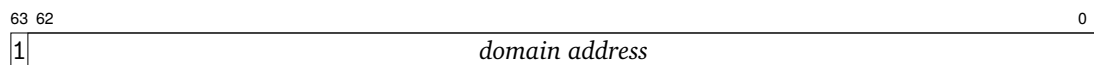
system thread address (saddr)

saddr is a read-only register containing the unique system-wide topological address for this thread. the system-wide topological address is used as a target for inter-thread virtual interrupts. this address is uniquely assigned during system initialization.

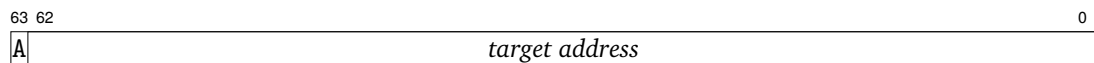


system domain address (sdomain)

`sdomain` is a read-only register containing the system-wide domain address for this thread. the system-wide domain address is used as a target for inter-thread virtual interrupts. this address can alias. this address is assigned during context switching.

**system target address (starget)**

starget is a read-write register containing the system-wide target address of a thread to send a virtual interrupt. all-zeros is the *boot service processor*. all-ones is the *broadcast address*. zero in the A field is a *topological address*. one in the A field is a *domain address*.



system message (smessage)

smmessage is a write-only register that causes an edge-triggered virtual interrupt to be queued to the thread set in `starget`. virtual interrupts are marked pending in the V field of the `ttstatus` register for the target thread.

system timer (stimer)

`stimer` is a read-write register containing the deadline timer for this thread. when the system clock reaches the value in the register, a *timer interrupt* is triggered. timer interrupts are marked pending in the T field of the `tstatus` register.

system interrupt enable (sie)

sie is a read-write register that contains interrupt enable flags for 32 wired interrupt pins.

system interrupt pending (sip)

sip is a read-only register that contains interrupt pending flags for 32 wired interrupt pins.

2.2.3. privileged debug registers

the section describes the privileged debug registers.

debug status (dstatus)

`dstatus` is a read-only register containing status bits for debug monitor exceptions.



no.	name	description
1 << 0	S	stopping instruction
1 << 1	F	fetch monitor address
1 << 2	R	load monitor address
1 << 3	W	store monitor address

Table 2.10.: debug status fields

debug cycle counter (dcycle)

dcycle is a read-only register containing the number of cycles retired since power on.

debug instruction counter (dinst)

dinst is a read-only register containing the number of instructions retired since power on.

debug stop instruction (dstop)

dstop is a read-write register containing an instruction number to halt execution on and raise a *debug monitor exception*.

debug monitor fetch address (dfetch)

dfetch is a read-write register containing a memory fetch address to monitor for and halt execution on and raise a *debug monitor exception* for a matching memory fetch address.

debug monitor read address (dread)

dread is a read-write register containing the memory load address to monitor for and halt execution on and raise a *debug monitor exception* for a matching memory load address.

debug monitor write address (dwrite)

`dwwrite` is a read-write register containing the memory store address to monitor for and halt execution on and raise a *debug monitor exception* for a matching memory store address.

2.3. Capability extension

the architecture provides an optional capabilities extension that includes capability registers controlling permissions and capabilities enabled by *page table color* bits in page table entries.

capability registers can be read and written using the `sysread`, `syswrite`, `sysset`, and `sysclear` instructions. the presence of capability registers is indicated by `sfeature.C` and they are available when `scontrol.C` is enabled and the associated executable page table colors are configured in `colorcap`.

the following tables list the privileged capability system parameters:

name	description
<i>ncol</i>	the number of architectural colors

Table 2.11.: capability parameters

capability registers

the following tables list the privileged capability system registers:

no.	name	description
color permission registers		
0x40	colorread	color read permissions
0x41	colorwrite	color write permissions
0x42	colorexec	color exec permissions
color capability registers		
0x43	colorsys	color system capabilities
0x44	colordom	color domain capabilities
0x45	colorcap	color capability capabilities
color matrix registers		
0x100...0x10f	colormatread[<i>n</i>]	color matrix read permissions
0x200...0x20f	colormatwrite[<i>n</i>]	color matrix write permissions
0x300...0x30f	colormatexec[<i>n</i>]	color matrix exec permissions

Table 2.12.: capability registers

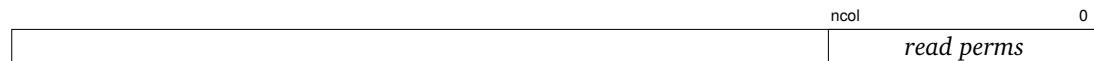
2. System

2.3.1. color permission registers

this section describes the *color permission registers*. the *color permission registers* are capability registers that protect access to memory based on *page table colors*. when the capabilities extension is enabled the *color permission registers* are combined with page table permissions to limit maximum *read*, *write*, or *execute* permission based on page table colors.

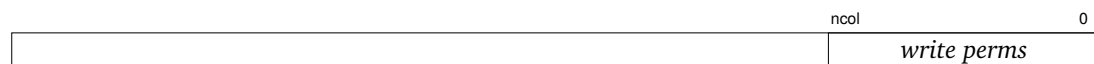
color read permissions (colorread)

colorread is a read-write register containing an array of *ncol* bits containing read permissions for *color-x* that are used to check loads from pages that have page table entries with matching page table colors.



color write permissions (colorwrite)

colorwrite is a read-write register containing an array of *ncol* bits containing write permissions for *color-x* that are used to check stores to pages that have page table entries with matching page table colors.



color exec permissions (colorexec)

colorexec is a read-write register containing an array of *ncol* bits containing execute permissions for *color-x* that are used to check execution in pages that have page table entries with matching page table colors.



2. System

2.3.2. color capability registers

this section describes the *color capability registers*. the *color capability registers* are capability registers that describe access to privileged system registers and instructions based on *page table colors* and register or instruction category. when the capabilities extension is enabled the *color capability registers* limit access to privileged system registers and instructions based on the system register categories: *system*, *domain*, and *capability*.

color system capabilities (colorsys)

colorsys is a read-write register containing an array of *ncol* bits containing system capabilities for *color-x* that are used to check access to privileged trap registers, privileged system registers and privileged debug registers from executable pages that have page table entries with a matching page table color.



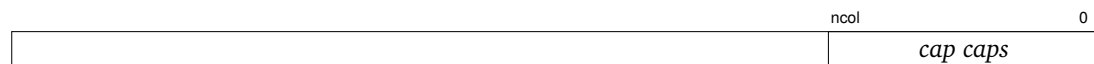
color domain capabilities (colordom)

colordom is a read-write register containing an array of *ncol* bits containing domain capabilities for *color-x* that are used to check access to privileged domain registers from executable pages that have page table entries with a matching page table color.



color capability capabilities (colorcap)

colorcap is a read-write register containing an array of *ncol* bits containing capability capabilities for *color-x* that are used to check access to privileged capability registers from executable pages that have page table entries with a matching page table color.



2. System

2.3.3. color matrix registers

this section describes the color matrix registers. the *color matrix registers* are permission registers that protect access to memory based on a combination of *source* and *destination* page table colors. when the capabilities extension is enabled the color matrix registers limit *read*, *write*, or *execute* permission based on executable pages that have page table entries with source *page table color* in memory accesses or branches to memory addresses in pages with matching *page table color* index.

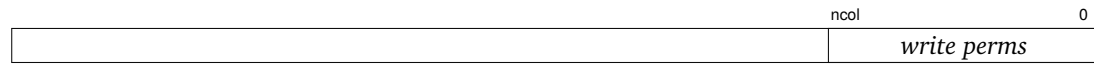
color matrix read permissions (colormatread[ncol])

colormatread[ncol] are *ncol* read-write registers containing an array of *ncol* bits with read permissions that are used to check loads from executable pages that have page table entries with a matching register number *page table color* to memory addresses in memory pages that have page table entries with a matching *page table color* array index.



color matrix write permissions (colormatwrite[ncol])

colormatwrite[ncol] are *ncol* read-write registers containing an array of *ncol* bits with write permissions that are used to check stores from executable pages that have page table entries with a matching register number *page table color* to memory addresses in memory pages that have page table entries with a matching *page table color* array index.



color matrix exec permissions (colormatexec[ncol])

colormatexec[ncol] are *ncol* read-write registers containing an array of *ncol* bits with exec permissions that are used to check branches from executable pages that have page table entries with a matching register number *page table color* to branch addresses in executable pages that have page table entries with a matching *page table color* array index.



2.4. Domain extension

the architecture provides an optional virtual domain extension that includes registers for domain address translation, domain routing and context switching for system traps, domain traps, and capability traps. capability trap routing requires both the virtual domains and capabilities extensions to be present and enabled.

domain registers can be read and written using the `sysread`, `syswrite`, `sysset`, and `sysclear` instructions. the presence of domain registers is indicated by `sfeature.V` and they are available when `scontrol.V` is enabled.

the following tables list the privileged domain system parameters:

name	description
<code>ndom</code>	the number of architectural domains

Table 2.13.: domain parameters

domain registers

the following tables list the privileged domain system registers:

no.	name	description
domain status registers		
0x50	<code>dom</code>	domain number
domain routing registers		
0x1000...0x100f	<code>domsuper[n]</code>	domain supervisor default domain
0x2000...0x200f	<code>domdom[n]</code>	domain domain default domain
0x3000...0x300f	<code>domcap[n]</code>	domain capability default domain
domain context registers		
0x4000...0x400f	<code>domaddr[n]</code>	domain address vector
0x5000...0x500f	<code>domptr[n]</code>	domain page table root vector
0x6000...0x600f	<code>domthpc[n]</code>	domain trap handler program counter
0x7000...0x700f	<code>domthib[n]</code>	domain trap handler immediate base

Table 2.14.: domain registers

2. System

2.4.1. domain status registers

this section describes the *domain status registers*.

domain number (dom)

dom is a read-write register containing the number of the currently running domain. it is initialized to zero. when the domain extension feature is present and enabled in `scontrol.V` this register is visible to the supervisor.

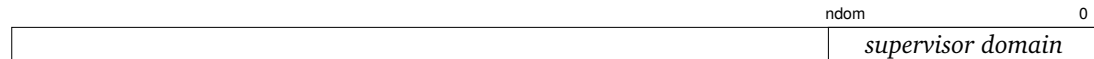


2.4.2. domain routing registers

this section describes the *domain routing registers* that control routing for context switching of supervisor traps, domain traps and capability traps.

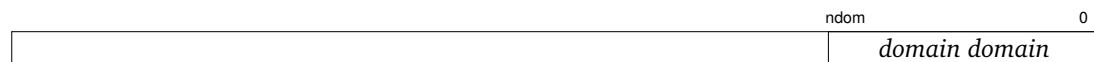
domain supervisor default domain vector (domsuper[ndom])

domsuper[ndom] are **ndom** read-write registers containing the domain number of the domain that supervisor traps are routed to. supervisor faults for the active domain will context switch to the domain in this register when the domain extension is enabled in `scontrol.V`.



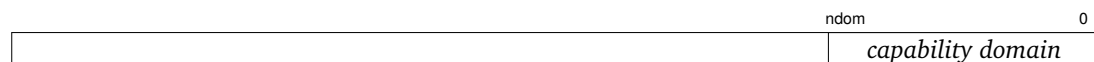
domain domain default domain vector (domdom[ndom])

domdom[ndom] are **ndom** read-write registers containing the domain number of the domain that domain traps are routed to. access to domain registers or instructions for the active domain will context switch to the domain in this register when the domain extension is enabled in `scontrol.V`.



domain capability default domain vector (domcap[ndom])

domcap[ndom] are **ndom** read-write registers containing the domain number of the domain that capability traps are routed to. access to capability registers for the active domain will context switch to the domain in this register when the domain and capabilities extensions are both enabled in `scontrol.V` and `scontrol.C`.



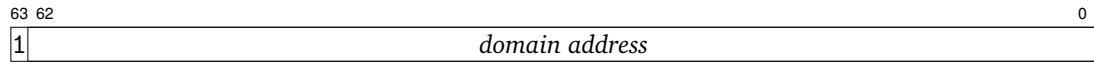
2. System

2.4.3. domain context registers

this section describes the *domain context registers* that control page translation and trap entry functions for domain context switching.

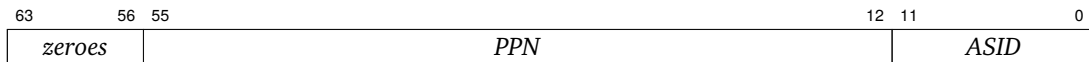
domain address vector (`domaddr[ndom]`)

`domaddr[ndom]` are *ndom* read-write registers containing an array of the domain addresses associated with each domain. when the domain feature is present and enabled in `scontrol.V` then domain fault context switches will cause the system to copy the domain specific value into `sdomain` for virtual interrupt routing in the domain.



domain page table root vector (`domptr[ndom]`)

`domptr[ndom]` are *ndom* read-write registers containing an array of the page table root for address translation in each domain. when the domain feature is present and is enabled in `scontrol.V` then domain fault context switches will cause the system to copy the domain specific value into `sptr` for address translation in the domain.



domain trap handler program counter vector (`domthpc[ndom]`)

`domthpc[ndom]` are *ndom* read-write registers containing an array of the addresses for the domain trap handler routines in each domain. the address is a virtual address interpreted in relation to the page table root for the domain. when the domain feature is present and is enabled in `scontrol.V` then domain fault context switches will cause the system to copy the domain specific value into `thpc` and set the *program counter* to that address.

domain trap handler immediate base vector (`domthib[ndom]`)

`domthib[ndom]` are *ndom* read-write registers containing an array of the addresses for the domain trap handler constants in each domain. the address is a virtual address interpreted in relation to the page table root for the domain. when the domain feature is present and is enabled in `scontrol.V` then domain fault context switches will cause the system to copy the domain specific value into `thib` and set the *immediate base* register to that address.

2.5. Address translation

the architecture provides for page-based virtual to physical address translation using a page table *trie structure* composed of index pages containing arrays of page table entries. a page walker reads the structure from the root pointer to leaf entries to translate virtual addresses into physical addresses. the architecture introduces the concept of a *translation address* which is an address boxed with an *address space prefix* (AS) designed to provide a canonical address form for user and supervisor virtual addresses as well as physical addresses.

2.5.1. page table structure

the section describes the dimensions for memory address translation and the *trie-based* page table structure pointed to by the *system page table root* (sptr) register.

Page Table Levels	Page Number Bits	Page Offset Bits	Virtual Address Bits	Page Index Entries	Page Sizes
4	9	12	44 — 48	512	4KiB, 2Mib, 1GiB

Table 2.15.: 64-bit page table dimensions

2.5.2. page table entries

page table entries are grouped into arrays within index pages which are selected by an index derived from a portion of the virtual address. each entry contains a physical page number along with several permission and metadata bits. the physical page number either points to the next page table level for *pointer* entries or the translated physical address for *leaf* entries.

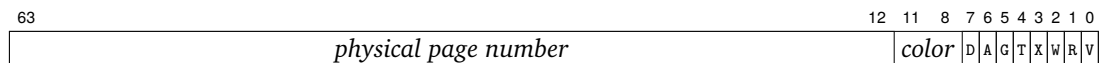


Figure 2.1.: 64-bit page table entry structure.

no.	name	description
1 << 0	V	valid
1 << 1	R	read
1 << 2	W	write
1 << 3	X	execute
1 << 4	T	translate
1 << 5	G	global
1 << 6	A	accessed
1 << 7	D	dirty

Table 2.16.: 64-bit page table entry fields

2. System

2.5.3. page table addresses

the page table translation system has three types of addresses: translation addresses, virtual addresses and physical addresses. page table translation addresses have a 1 — 4 bit address space prefix to allow them to contain user and supervisor virtual addresses as well as physical addresses. the page table translation and lookup virtual address structure is as follows:

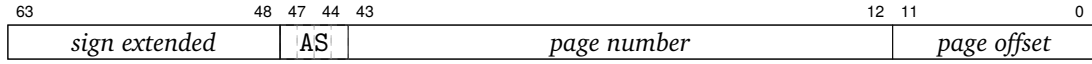


Figure 2.2.: 64-bit translation address structure.

no.	name
0b0.	user
0b10.	supervisor
0b110.	physical

Table 2.17.: 1—4 bit translation address space prefixes.

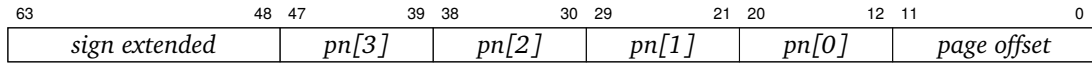


Figure 2.3.: 64-bit lookup virtual address structure.

2.5.4. page table translation

the page table walker performs page table lookups to translate virtual addresses into physical addresses. the page table walker reads the page table root pointer then walks the *trie-based* page table structure to find a leaf entry containing a physical page address.

page table entries with the *read*, *write* and *exec* permission bits clear are interpreted as *pointer* entries and contain a pointer to the next-level index page, otherwise they are interpreted as *leaf* entries with a final translation. entries with the *write* bit set must have the *read* bit set. entries without the *accessed* bit set will fault on read accesses. entries without the *dirty* bit set will fault on write accesses. the *global* bit prevents *sptr.ASID* from being associated with TLB entries. the *color* field is reserved for use by system software and the capabilities extension.

if the physical permissions feature *sfeature.P* is present and *scontrol.P* is enabled, then physical pages must be present in the page table with a self-mapping of their physical address prefixed with (*AS = physical*) leading to a leaf page table entry containing its own address. this is to allow physical page permissions to be checked. in addition to this, the physical self-mappings for physical pages containing page table pages must have the *translate* bit set.

the page table lookup and virtual to physical address translation process are as follows:

2. System

Algorithm 1 find page table entry for translation address

```

1: function FIND_PAGE(type, level, ppn, ta)
2:   shift  $\leftarrow level \times pn\_bits + po\_bits$                                 ▷ calculate page number shift
3:   tpn  $\leftarrow (ta \gg shift) \wedge ((1 \ll pn\_bits) - 1)$                     ▷ calculate translation page number
4:   pte  $\leftarrow \text{LOAD}((ppn \ll po\_bits) + tpn \times \text{sizeof}(PTE))$               ▷ load page table entry
5:   leaf  $\leftarrow pte.R \vee pte.W \vee pte.X$                                 ▷ page table entry node type
6:   sm  $\leftarrow R \vee W \vee X$                                               ▷ default self map page table entry
7:   if sfeature.P  $\wedge$  scontrol.P  $\wedge$  type = virtual then
8:     pa  $\leftarrow pte.ppn \ll po\_shift$ 
9:     (sm, level)  $\leftarrow \text{FIND\_PAGE}(\text{physical}, num\_levels - 1, sptr.ppn, pa)$     ▷ load self mapping
10:    if (leaf  $\wedge$  pte.ppn  $\neq$  sm.ppn)  $\vee (\neg leaf \wedge \neg sm.T)$  then
11:      raise fault                                                         ▷ invalid self mapping
12:    end if
13:  end if
14:  if  $\neg leaf \wedge level > 0$  then
15:    return FIND_PAGE(type, level - 1, pte.ppn, ta)                      ▷ follow pointer entry
16:  else if  $\neg leaf$  then
17:    raise fault                                                         ▷ leaf entry not found
18:  end if
19:  return (pte, sm, level)                                              ▷ return page table entry
20: end function

```

Algorithm 2 translate virtual address to physical address

```

1: function TRANSLATE(op, va)
2:   if  $\neg \text{CHECK\_CANONICAL}(va, va\_bits)$  then                                ▷ check address is sign-extended
3:     raise fault
4:   end if
5:   (pte, sm, level)  $\leftarrow \text{FIND\_PAGE}(\text{virtual}, num\_levels - 1, sptr.ppn, va)$     ▷ find page table entry
6:   mask  $\leftarrow ((1 \ll (level \times pn\_bits + po\_bits)) - 1)$ 
7:   caps  $\leftarrow (R \wedge \text{colorread}[pte.color]) \vee (W \wedge \text{colorwrite}[pte.color]) \vee (X \wedge \text{colorexec}[pte.color])$ 
8:   user  $\leftarrow \neg((va \gg (va\_bits - 1)) \wedge 1)$ 
9:   if  $(\neg pte.V) \vee (pte.W \wedge \neg pte.R)$  then
10:    raise fault                                                         ▷ invalid write must have read
11:  else if  $(op.R \wedge \neg pte.R) \vee (op.W \wedge \neg pte.W) \vee (op.X \wedge \neg pte.X)$  then
12:    raise fault                                                         ▷ invalid permissions
13:  else if  $(sm.R \wedge \neg pte.R) \vee (sm.W \wedge \neg pte.W) \vee (sm.X \wedge \neg pte.X)$  then
14:    raise fault                                                         ▷ invalid self map
15:  else if  $(cap.R \wedge pte.R) \vee (cap.W \wedge pte.W) \vee (cap.X \wedge pte.X)$  then
16:    raise fault                                                         ▷ invalid capabilities
17:  else if  $(op.R \wedge \neg pte.A) \vee (op.W \wedge \neg pte.D)$  then
18:    raise fault                                                         ▷ invalid accessed or dirty
19:  else if  $\neg tstatus.U \wedge \neg scontrol.E \wedge user \wedge op.X$  then
20:    raise fault                                                         ▷ invalid user page execute
21:  else if  $\neg tstatus.U \wedge \neg scontrol.A \wedge user \wedge op.(R \vee W)$  then
22:    raise fault                                                         ▷ invalid user page access
23:  else if  $(pte.ppn \ll po\_bits) \wedge mask \neq 0$  then
24:    raise fault                                                         ▷ invalid superpage alignment
25:  end if
26:  return  $(pte.ppn \ll po\_bits) + (va \wedge mask)$ 
27: end function

```

3. Instructions

3.1. Instruction listing — 16-bit

3.1.1. break

15	7	6	2	1	0
<i>imm9</i>			<i>opcode</i>	<i>size</i>	
uimm			00000	00	

break *uimm9*

the *break* instruction causes a debugger trap. *program counter* and trap cause are saved to privileged registers for the operating system to dispatch to a debugger and the *program counter* is set to a trap vector address.

3.1.2. j

15	7	6	2	1	0
<i>imm9</i>			<i>opcode</i>	<i>size</i>	
simm			00001	00	

j *simm9* × 2

the *j* or *jump* instruction is an unconditional branch instruction that adds a relative immediate address to the *program counter*. the resulting *program counter* address is [*pc* + *simm9* × 2].

$$pc = pc + simm9 * 2$$

3.1.3. b

15	7	6	2	1	0
<i>imm9</i>			<i>opcode</i>	<i>size</i>	
simm			00010	00	

b *simm9* × 2

the *b* or *branch* instruction is a conditional branch instruction that adds a relative immediate address to the *program counter*. if the *flag* register has been set by a compare instruction, the resulting *program counter* address is [*pc* + *simm9* × 2], otherwise the *program counter* is advanced normally.

```
if flag:
    pc = pc + simm9 * 2
```

3. Instructions

3.1.4. ibj

15	7	6	2	1	0
<i>imm9</i>			<i>opcode</i>	<i>size</i>	
simm			00011	00	

ibj $\text{simm9} \times 64$

the *ibj* or *immediate-block-jump* instruction adds a 64-bit relative address to the *immediate base* register. the resulting *immediate base* address is $[ib + \text{simm9} \times 64]$.

$$ib = ib + \text{simm9} * 64$$

3.1.5. link

15	13	12	7	6	2	1	0
<i>rc</i>		<i>imm6</i>		<i>opcode</i>	<i>size</i>		
fun		uimm		00100	00		

link.i64 $\text{fun3}, \text{ib64}(\text{uimm6})$

the *link* instruction loads a 64-bit constant addressed by $[ib + \text{uimm6} \times 8]$ containing a *i32x2* relative address vector, which it adds it to (pc, ib) , conditionally subtracts the source link register (*r6* or *r7*), then conditionally adds the difference to the destination link register (*r6* or *r7*), depending on the value of *fun3*. the type of linkage in *fun3* can be one of:

value	mnemonic	description	link-type	dst-link	src-link
0	jib	jump	jump	-	-
1	-	-	-	-	-
2	jalib	jump-and-link	call	r6	-
3	jalib	jump-and-link	call	r7	-
4	jtlb	jump-to-link	ret	-	r6
5	jtlb	jump-to-link	ret	-	r7
6	jalaib	jump-and-link-add	tail	r6	-
7	jalaib	jump-and-link-add	tail	r7	-

Table 3.1.: Link instruction functions

```

lr = 0b110 + (fun3 & 1)
cval = const-mem<i32x2>[ib + uimm6 * 8]
dval = fun3 == jalaib ? cval + auth-decrypt(reg<i32x2>[lr]) : cval
lval = fun3 == jtlb ?      auth-decrypt(reg<i32x2>[lr]) : 0
(dpc,dib) = dval
(lpc,lib) = lval
pc = pc + dpc - lpc
ib = ib + dib - lib
if fun3 == jalib or fun3 == jalaib:
    reg[lr] = auth-encrypt(reg<i32x2>(dpc,dib))

```

the *auth-encrypt* and *auth-decrypt* functions are placeholders for authenticated encryption, and decryption of relative address vectors. authentication failures should generate a trap for the operating system to dispatch to a control flow integrity trap handler.

3. Instructions

3.1.6. movh

15	13	12	7	6	2	1	0
<i>rc</i>	<i>imm6</i>			<i>opcode</i>	<i>size</i>		
rc	uimm			00101	00		

movh.i64 *rc*, *ib32(uimm6)*

the *movh* or *move-half-immediate-block* instruction loads a 32-bit constant addressed by $[ib + uimm6 \times 4]$, which it sign-extends to 64-bits, then saves to the *rc* register.

$$\text{reg}[rc] = \text{const-mem}<i32>[ib + uimm6 * 4]$$

3.1.7. movw

15	13	12	7	6	2	1	0
<i>rc</i>	<i>imm6</i>			<i>opcode</i>	<i>size</i>		
rc	uimm			00110	00		

movw.i64 *rc*, *ib64(uimm6)*

the *movw* or *move-word-immediate-block* instruction loads a 64-bit constant addressed by $[ib + uimm6 \times 8]$ then saves it to the *rc* register.

$$\text{reg}[rc] = \text{const-mem}<i64>[ib + uimm6 * 8]$$

3.1.8. movi

15	13	12	7	6	2	1	0
<i>rc</i>	<i>imm6</i>			<i>opcode</i>	<i>size</i>		
rc	simm			00111	00		

movi.i64 *rc*, *simm6*

the *movi* or *move-immediate* instruction sign-extends the immediate value in *simm6* then saves the result in the *rc* register.

$$\text{reg}[rc] = \text{simm6}$$

3.1.9. addi

15	13	12	7	6	2	1	0
<i>rc</i>	<i>imm6</i>			<i>opcode</i>	<i>size</i>		
rc	simm			01000	00		

addi.i64 *rc*, *simm6*

the *addi* or *add-immediate* instruction sign-extends the immediate value in *simm6* then adds it to the *rc* register.

$$\text{reg}[rc] = \text{reg}[rc] + \text{simm6}$$

3. Instructions

3.1.10. srli

15	13	12	7	6	2	1	0
<i>rc</i>	<i>imm6</i>			<i>opcode</i>	<i>size</i>		
rc	uimm			01001	00		

srli.i64 *rc*, *uimm6*

the *srli* or *shift-right-logical-immediate* instruction performs a logical right shift by *uimm6* bits of the value in the *rc* register. zeros are copied into the left most bits.

$$\text{reg}[\text{rc}] = \text{reg}\langle\text{u64}\rangle[\text{rc}] \gg \text{uimm6}$$

3.1.11. srai

15	13	12	7	6	2	1	0
<i>rc</i>	<i>imm6</i>			<i>opcode</i>	<i>size</i>		
rc	uimm			01010	00		

srai.i64 *rc*, *uimm6*

the *srai* or *shift-right-arithmetic-immediate* instruction performs an arithmetic right shift by *uimm6* bits of the value in the *rc* register. the sign is copied into the left most bits.

$$\text{reg}[\text{rc}] = \text{reg}\langle\text{i64}\rangle[\text{rc}] \gg \text{uimm6}$$

3.1.12. slli

15	13	12	7	6	2	1	0
<i>rc</i>	<i>imm6</i>			<i>opcode</i>	<i>size</i>		
rc	uimm			01011	00		

slli.i64 *rc*, *uimm6*

the *slli* or *shift-left-logical-immediate* instruction performs a logical left shift by *uimm6* bits of the value in the *rc* register. zeros are copied into the right most bits.

$$\text{reg}[\text{rc}] = \text{reg}[\text{rc}] \ll \text{uimm6}$$

3.1.13. addh

15	13	12	7	6	2	1	0
<i>rc</i>	<i>imm6</i>			<i>opcode</i>	<i>size</i>		
rc	uimm			01100	00		

addh.i64 *rc*, *ib32*(*uimm6*)

the *addh* or *add-half-immediate-block* instruction loads a 32-bit constant addressed by [*ib* + *uimm6* × 4], which it sign-extends to 64-bits, then adds to the *rc* register.

$$\text{reg}[\text{rc}] = \text{reg}[\text{rc}] + \text{const-mem}\langle\text{i32}\rangle[\text{ib} + \text{uimm6} * 4]$$

3. Instructions

3.1.14. leapc

15	13	12	7	6	2	1	0
<i>rc</i>	<i>imm6</i>			<i>opcode</i>	<i>size</i>		
rc	uimm			01101	00		

leapc.i64 *rc*, *ib*32(*uimm6*)(*pc*)

the *leapc* or *load-effective-address-pc* instruction loads a 32-bit constant addressed by [*ib* + *uimm6* × 4], which it sign-extends to 64-bits, adds it to the *program counter*, and saves the result in the *rc* register.

$$\text{reg}[\text{rc}] = \text{pc} + \text{const-mem}\langle\text{i32}\rangle[\text{ib} + \text{uimm6} * 4]$$

3.1.15. loadpc

15	13	12	7	6	2	1	0
<i>rc</i>	<i>imm6</i>			<i>opcode</i>	<i>size</i>		
rc	uimm			01110	00		

loadpc.i64 *rc*, *ib*32(*uimm6*)(*pc*)

the *loadpc* instruction loads a 32-bit constant addressed by [*ib* + *uimm6* × 4] which it sign-extends to 64-bit, adds it to the *program counter* to form an address, then loads a 64-bit value from memory at that address and saves the result in the *rc* register.

$$\text{reg}[\text{rc}] = \text{mem}\langle\text{i64}\rangle[\text{pc} + \text{const-mem}\langle\text{i32}\rangle[\text{ib} + \text{uimm6} * 4]]$$

3.1.16. storepc

15	13	12	7	6	2	1	0
<i>rc</i>	<i>imm6</i>			<i>opcode</i>	<i>size</i>		
rc	uimm			01111	00		

storepc.i64 *rc*, *ib*32(*uimm6*)(*pc*)

the *storepc* instruction loads a 32-bit constant addressed by [*ib* + *uimm6* × 4] which it sign-extends to 64-bit, adds it to the *program counter* to form an address, then stores to memory at that address a 64-bit value from the *rc* register.

$$\text{mem}\langle\text{i64}\rangle[\text{pc} + \text{const-mem}\langle\text{i32}\rangle[\text{ib} + \text{uimm6} * 4]] = \text{reg}[\text{rc}]$$

3.1.17. load

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>imm3</i>		<i>opcode</i>		<i>size</i>			
rc	rb	uimm		10000		00			

load.i64 *rc*, (*uimm3* × 8)(*rb*)

the *load* instruction computes the address [*rb* + *uimm3* × 8] then loads a 64-bit value from memory at that address and saves the result in the *rc* register.

$$\text{reg}[\text{rc}] = \text{mem}\langle\text{i64}\rangle[\text{reg}[\text{rb}] + \text{uimm3} * 8]$$

3. Instructions

3.1.18. store

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>imm3</i>	<i>opcode</i>	<i>size</i>					
rc	rb	uimm	10001	00	store.i64 <i>rc</i> , (<i>uimm3</i> × 8)(<i>rb</i>)				

the *store* instruction computes the address $[rb + uimm3 \times 8]$ then stores a 64-bit value to memory at that address containing a 64-bit value from the *rc* register.

$$\text{mem}\langle i64 \rangle[\text{reg}[rb] + uimm3 * 8] = \text{reg}[rc]$$

3.1.19. compare

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>imm3</i>	<i>opcode</i>	<i>size</i>					
rc	rb	fun	10010	00	cmp.i64 <i>rc</i> , <i>rb</i> , <i>fun3</i>				

the *compare* instruction performs a comparison between the value in *rb* and *rc* then saves the result in the *flag* register. the *compare* opcode is also used to perform conditional move whereby the *rb* register is copied into the *rc* if the *flag* register is set. the type of comparison in *fun3* can be one of:

value	mnemonic	description
0	<i>lt</i>	<i>less than (signed)</i>
1	<i>ge</i>	<i>greater or equal (signed)</i>
2	<i>eq</i>	<i>equal</i>
3	<i>ne</i>	<i>not equal</i>
4	<i>ltu</i>	<i>less than (unsigned)</i>
5	<i>geu</i>	<i>greater or equal (unsigned)</i>
6	<i>cmov</i>	<i>conditional move</i>
7	<i>ncmov</i>	<i>negated conditional move</i>

Table 3.2.: Compare instruction functions

```

match fun3
| lt    -> flag = reg<i64>[rc] < reg<i64>[rb]
| ge    -> flag = reg<i64>[rc] >= reg<i64>[rb]
| eq    -> flag = reg[rc] == reg[rb]
| ne    -> flag = reg[rc] != reg[rb]
| ltu   -> flag = reg<u64>[rc] < reg<u64>[rb]
| geu   -> flag = reg<u64>[rc] >= reg<u64>[rb]
| cmov  -> if flag:
            reg[rc] = reg[rb]
| ncmov -> if not flag:
            reg[rc] = reg[rb]

```

3. Instructions

3.1.20. logic

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>imm3</i>	<i>opcode</i>	<i>size</i>					
rc	rb	fun	10011	00					

logic.i64 *rc,rb,fun3*

the *logic* instruction performs a logic operation on the value in the *rb* register then stores the result in the *rc* register. the type of logic operations in *fun3* can be one of:

value	mnemonic	description
0	<i>mv</i>	<i>move</i>
1	<i>not</i>	<i>logical not</i>
2	<i>neg</i>	<i>negate</i>
3	<i>bswap</i>	<i>bswap</i>
4	<i>ctz</i>	<i>count trailing zeros</i>
5	<i>clz</i>	<i>count leading zeros</i>
6	<i>ctpop</i>	<i>count population</i>
7	<i>sext</i>	<i>sign extend</i>

Table 3.3.: Logic instruction functions

```
match fun3
| mov  -> reg[rc] = reg[rb]
| not  -> reg[rc] = ~reg[rb]
| neg  -> reg[rc] = -reg[rb]
| bswap -> reg[rc] = bswap(reg[rb])
| ctz   -> reg[rc] = ctz(reg[rb])
| clz   -> reg[rc] = clz(reg[rb])
| ctpop -> reg[rc] = ctpop(reg[rb])
| sext  -> reg[rc] = sext(reg[rb])
```

3.1.21. pin

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>ra</i>	<i>opcode</i>	<i>size</i>					
rc	rb	ra	10100	00					

pin.i64 *rc,rb,ra*

the *pin* or *pack-indirect* instruction packs two absolute addresses as an *i32x2* (*pc,ib*) relative address vector. the register *ra* is subtracted from the *program counter*, and the register *rb* is subtracted from the *immediate base* register, and the results are packed into an *i32x2* relative address vector and saved to the register *rc*.

```
lpc  = int<i32>(pc - reg[ra])
lib  = int<i32>(ib - reg[rb])
lval = (lpc,lib)
reg[rc] = auth-encrypt(lval)
```

3. Instructions

3.1.22. and

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>ra</i>	<i>opcode</i>				<i>size</i>		
rc	rb	ra	10101				00		

and.i64 *rc,rb,ra*

the *and* instruction performs a *logical-and* of the register *rb* and the register *ra* and saves the result in the register *rc*.

$$\text{reg}[rc] = \text{reg}[rb] \ \& \ \text{reg}[ra]$$

3.1.23. or

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>ra</i>	<i>opcode</i>				<i>size</i>		
rc	rb	ra	10110				00		

or.i64 *rc,rb,ra*

the *or* instruction performs a *logical-or* of the register *rb* and the register *ra* and saves the result in the register *rc*.

$$\text{reg}[rc] = \text{reg}[rb] \ | \ \text{reg}[ra]$$

3.1.24. xor

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>ra</i>	<i>opcode</i>				<i>size</i>		
rc	rb	ra	10111				00		

xor.i64 *rc,rb,ra*

the *xor* instruction performs a *logical-exclusive-or* of the register *rb* and the register *ra* and saves the result in the register *rc*.

$$\text{reg}[rc] = \text{reg}[rb] \ \wedge \ \text{reg}[ra]$$

3.1.25. add

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>ra</i>	<i>opcode</i>				<i>size</i>		
rc	rb	ra	11000				00		

add.i64 *rc,rb,ra*

the *add* instruction adds the *rb* register to the *ra* register and saves the result in the *rc* register.

$$\text{reg}[rc] = \text{reg}[rb] \ + \ \text{reg}[ra]$$

3. Instructions

3.1.26. srl

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>ra</i>	<i>opcode</i>				<i>size</i>		
rc	rb	ra	11001				00		

srl.i64 *rc,rb,ra*

the *srl* or *shift-right-logical* instruction performs a logical right shift of the value in the *rb* register by the number of bits in register *ra* then saves the result in the *rc* register. zeros are copied into the right most bits.

$$\text{reg}[rc] = \text{reg}\langle u64 \rangle[rb] \gg \text{reg}[ra]$$

3.1.27. sra

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>ra</i>	<i>opcode</i>				<i>size</i>		
rc	rb	ra	11010				00		

sra.i64 *rc,rb,ra*

the *sra* or *shift-right-arithmetic* instruction performs an arithmetic right shift of the value in the *rb* register by the number of bits in register *ra* then saves the result in the *rc* register. sign is copied into the right most bits.

$$\text{reg}[rc] = \text{reg}\langle i64 \rangle[rb] \gg \text{reg}[ra]$$

3.1.28. sll

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>ra</i>	<i>opcode</i>				<i>size</i>		
rc	rb	ra	11011				00		

sll.i64 *rc,rb,ra*

the *sll* or *shift-left-logical* instruction performs a logical left shift of the value in the *rb* register by the number of bits in register *ra* then saves the result in the *rc* register.

$$\text{reg}[rc] = \text{reg}[rb] \ll \text{reg}[ra]$$

3.1.29. sub

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>ra</i>	<i>opcode</i>				<i>size</i>		
rc	rb	ra	11100				00		

sub.i64 *rc,rb,ra*

the *sub* instruction subtracts the *ra* register from the *rb* register and saves the result in the *rc* register.

$$\text{reg}[rc] = \text{reg}[rb] - \text{reg}[ra]$$

3. Instructions

3.1.30. mul

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>ra</i>	<i>opcode</i>				<i>size</i>		
rc	rb	ra	11101				00		

mul.i64 *rc,rb,ra*

the *mul* instruction performs signed multiplication of the *rb* register with the *ra* register and saves the result in the *rc* register.

```
reg[rc] = reg[rb] * reg[ra]
```

3.1.31. div

15	13	12	10	9	7	6	2	1	0
<i>rc</i>	<i>rb</i>	<i>ra</i>	<i>opcode</i>				<i>size</i>		
rc	rb	ra	11110				00		

div.i64 *rc,rb,ra*

the *div* instruction performs signed division of the *rb* register by the *ra* register and saves the result in the *rc* register. division by zero causes the flag to be set and zero to be stored in the *rc* register. a subsequent branch can handle the division by zero.

```
flag = reg[ra] == 0
if flag:
    reg[rc] = 0
else:
    reg[rc] = reg[rb] / reg[ra]
```

3.1.32. illegal

15	7	6	2	1	0
<i>imm9</i>			<i>opcode</i>	<i>size</i>	
uimm			11111	00	

illegal *uimm9*

the *illegal* instruction causes an illegal instruction trap. *program counter* and trap cause are saved to privileged registers for the operating system to dispatch to an illegal instruction handler and the *program counter* is set to a trap vector address.

4. Assembly

4.1. Introduction

this glyph assembly language reference begins with an introduction to assembler and linker concepts and terminology, followed by sections describing the glyph assembler directives, and pseudo-instruction aliases. section 3 contains a complete listing of instruction.

4.2. Concepts

this section covers assembler high level concepts required to understand the concepts involved in assembling and linking executable code from source files. the terminology used in this section is applicable to the PE/COFF, ELF and Mach-O file formats.

4.2.1. assembly

assembly files contains assembly language directives, macros and instructions describing program code and data. they can be handwritten or emitted by a compiler. an assembly file is the input file to the assembler and the output from the assembler is an object file.

4.2.2. object

object files contain compiled relocatable object code and data emitted by the assembler. an object file cannot be run, rather it is used as input to the linker in a program linking step which combines them to produce an executable file or shared library.

4.2.3. archive

archive files contain collections of relocatable object files and are typically referred to as static libraries. archive files use a simple format that appends together a set of object files with an index listing the object files contained in the archive. thin archive files just contain the index listing the object files without any associated data.

4.2.4. executable

executable files contain compiled relocatable object code and data that has been linked together by a linker in a program linking step using multiple object files, archive files and shared libraries as input. executable files can be statically or dynamically linked. dynamically linked executables files express dependencies on shared libraries.

4.2.5. shared library

shared library files contain compiled relocatable object code and data that have been linked together by a linker in a program linking step using multiple object files, archive files and shared libraries as input. shared libraries are dynamically linked by a runtime linker, and they express dependencies on other shared libraries which need to be loaded at runtime.

4.2.6. section

a section is a name for a region of code or data in an object file, executable file or shared library file. the files can be made up of multiple sections where each section corresponds to several types of executable code or data. this list contains the most common types:

- `.text` is a read-only section containing executable code
- `.const` is a read-only section containing immediate blocks
- `.data` is a read-write section containing global or static variables
- `.rodata` is a read-only section containing read-only variables
- `.bss` is a read-write section containing uninitialized data

4.2.7. segment

segments are loadable regions of code or data in an an executable or shared library. segments describe virtual addresses, file offsets and memory access permissions for mapped sections. in ELF, a segment can map to one or more sections. in PE/COFF sections are mapped directly. in Mach-O, sections are contained within a small set of specific segment types.

4.2.8. symbol

symbols are metadata table entries that contains a name with a mapping to an address. symbols are present in object files and dynamic symbols are present in shared libraries and executables. they can be referred to in relocation entries and debugging metadata.

4.2.9. relocation

relocations are metadata table entries used to update relocatable addresses during linking. relocations contains a type, a file offset pointing to text or data, and a pointer to a symbol name whose address needs to be updated during the linking step. relocations can be present in object files and dynamic relocations are present in shared libraries and executables.

4.2.10. program linking

program linking is the process of combining multiple relocatable object files by merging and aligning sections, resolving symbol references across files, assigning final symbol addresses, applying relocation fixups using relocation entries, and adding debug metadata.

4. Assembly

4.3. Directives

the assembler implements a number of directives that control the assembly of instructions into object files. these directives are based on the AT&T System V assembler [2] and the GNU assembler [5], with some additions. they provide the ability to include arbitrary data, align data, export symbols, switch sections, define constants, and emit metadata.

the following table lists glyph assembler directives:

Directive	Arguments	Description
Data directives		
<code>.byte</code>	<i>expression-list</i>	8-bit comma separated words
<code>.short</code>	<i>expression-list</i>	16-bit comma separated words
<code>.long</code>	<i>expression-list</i>	32-bit comma separated words
<code>.quad</code>	<i>expression-list</i>	64-bit comma separated words
<code>.octa</code>	<i>expression-list</i>	128-bit comma separated words
<code>.string</code>	<i>"string"</i>	emit string
<code>.zero</code>	<i>integer</i>	emit zeroes
Alignment directives		
<code>.align</code>	<i>pow2</i> [<i>pad_val=0</i>] [<i>max</i>]	align to power of 2
<code>.balign</code>	<i>bytes</i> [<i>pad_val=0</i>]	byte align
Symbol directives		
<code>.globl</code>	<i>symbol_name, const_name</i>	emit symbol (<i>global scope</i>)
<code>.local</code>	<i>symbol_name, const_name</i>	emit symbol (<i>local scope</i>)
Section directives		
<code>.text</code>	<i>symbol_name, size, align</i> <i>section_name</i>	emit <code>.text</code> section or make current
<code>.const</code>		emit <code>.const</code> section or make current
<code>.data</code>		emit <code>.data</code> section or make current
<code>.rodata</code>		emit <code>.rodata</code> section or make current
<code>.bss</code>		emit <code>.bss</code> section or make current
<code>.common</code>		emit common object to <code>.bss</code> section
<code>.section</code>		emit section (default <code>.text</code>) or make current
Miscellaneous directives		
<code>.equ</code>	<i>name, value</i>	constant definition
<code>.file</code>	<i>"filename"</i>	emit filename symbol
<code>.ident</code>	<i>"string"</i>	emit identification string
<code>.size</code>	<i>symbol, symbol</i>	emit symbol size
<code>.type</code>	<i>symbol, @function</i>	emit symbol type

Table 4.1.: Assembler directives

4. Assembly

4.4. Pseudo-instructions

the assembler implements a number of convenience psuedo-instruction aliases that are formed from regular instructions, but have implicit or deduced arguments.

the following table lists glyph assembler pseudo instruction aliases:

Pseudo-instruction	Expansion	Description
<code>nop</code>	<code>or.i64 r0,r0,r0</code>	no-operation
<code>li rc, expression</code>	(several expansions)	load immediate
<code>la rc, symbol</code>	(several expansions)	load address
<code>call symbol</code>	<code>jalib.i64 ibcall(text-label,const-label)</code>	procedure call
<code>ret</code>	<code>jtlib.i64 ibret(text-label,const-label)</code>	procedure return
<code>jib.i64 ib64(uimm6)</code>	<code>link.i64 ib64(uimm6), jib</code>	jump-immmediate-block
<code>jalib.i64 rc, ib64(uimm6)</code>	<code>link.i64 rc, ib64(uimm6), jalib</code>	jump-and-link-immmediate-block
<code>jtlib.i64 rc, ib64(uimm6)</code>	<code>link.i64 rc, ib64(uimm6), jtlib</code>	jump-to-link-immmediate-block
<code>jalaib.i64 rc, ib64(uimm6)</code>	<code>link.i64 rc, ib64(uimm6), jalaib</code>	jump-and-link-add-immmediate-block
<code>cmp.lt.i64 rc, rb</code>	<code>compare.i64 rc, rb, lt</code>	compare less than (signed)
<code>cmp.gt.i64 rc, rb</code>	<code>compare.i64 rb, rc, lt</code>	compare greater than (signed)
<code>cmp.le.i64 rc, rb</code>	<code>compare.i64 rb, rc, ge</code>	compare less or equal (signed)
<code>cmp.ge.i64 rc, rb</code>	<code>compare.i64 rc, rb, ge</code>	compare greater or equal (signed)
<code>cmp.eq.i64 rc, rb</code>	<code>compare.i64 rc, rb, eq</code>	compare equal
<code>cmp.ne.i64 rc, rb</code>	<code>compare.i64 rc, rb, ne</code>	compare not equal
<code>cmp.ltu.i64 rc, rb</code>	<code>compare.i64 rc, rb, ltu</code>	compare less than (unsigned)
<code>cmp.gtu.i64 rc, rb</code>	<code>compare.i64 rb, rc, ltu</code>	compare greater than (unsigned)
<code>cmp.leu.i64 rc, rb</code>	<code>compare.i64 rb, rc, geu</code>	compare less or equal (unsigned)
<code>cmp.geu.i64 rc, rb</code>	<code>compare.i64 rc, rb, geu</code>	compare greater or equal (unsigned)
<code>cmov.i64 rc, rb</code>	<code>compare.i64 rc, rb, cmov</code>	conditional move
<code>ncmov.i64 rc, rb</code>	<code>compare.i64 rc, rb, ncmov</code>	negated conditional move
<code>mov.i64 rc, rb</code>	<code>logic.i64 rc, rb, mov</code>	copy register
<code>not.i64 rc, rb</code>	<code>logic.i64 rc, rb, not</code>	logical not
<code>neg.i64 rc, rb</code>	<code>logic.i64 rc, rb, neg</code>	signed negate
<code>bswap.i64 rc, rb</code>	<code>logic.i64 rc, rb, bswap</code>	byte swap
<code>ctz.i64 rc, rb</code>	<code>logic.i64 rc, rb, ctz</code>	count trailing zeros
<code>clz.i64 rc, rb</code>	<code>logic.i64 rc, rb, clz</code>	count leading zeroes
<code>ctpop.i64 rc, rb</code>	<code>logic.i64 rc, rb, ctpop</code>	count population
<code>sext.i64 rc, rb</code>	<code>logic.i64 rc, rb, sext</code>	sign extend

Table 4.2.: Pseudo instructions

4.5. Calling convention

4.5.1. calling convention — 16-bit

the 16-bit instruction packet, while intended to be used in conjunction with the larger opcodes, is designed as a complete subset, so there is an ABI variant that targets a subset of the instruction set architecture that only uses the 16-bit opcodes.

the register assignment for the 16-bit subset was chosen with this rationale:

- 2 blocks of 4 contiguous non-volatile *callee-save* and volatile *caller-save* registers.
- 3 special registers, 2 argument registers, 1 temporary register, and 3 save registers.
- 3 save registers to avoid excessive spilling around function calls.
- 1 temporary register to avoid spilling arguments to free a temporary.

the calling convention for the 16-bit subset is as follows:

- *immediate base* `ib` is set by `call` instructions and must point to a valid immediate block on function entry. function symbols are exported with two labels; one in the `.text` section, and one in the `.const` section. *immediate base* must be restored to the entry value in the function *epilogue* before it can be restored by `ret`.
- *argument registers* `a0` and `a1` are used for the first two arguments, and the remaining arguments are passed on the stack. *return value* is places in `a0` and `a1`, *temporary register* `t0` is a volatile register, and *frame pointer* (if enabled) uses `s0`. there are two more non-volatile *callee-save* registers, `s1` and `s2`.

the following table outlines the 16-bit register allocation showing register name alias, description, and non-volatile *callee-save* or volatile *caller-save* status.

name	alias	description	save
<code>ib</code>		immediate base	callee
<code>r0</code>	<code>sp</code>	stack pointer	callee
<code>r1</code>	<code>s0/fp</code>	saved register 0 / frame pointer	callee
<code>r2</code>	<code>s1</code>	saved register 1	callee
<code>r3</code>	<code>s2</code>	saved register 2	callee
<code>r4</code>	<code>a0</code>	argument register 0	caller
<code>r5</code>	<code>a1</code>	argument register 1	caller
<code>r6</code>	<code>t0</code>	temporary register 0	caller
<code>r7</code>	<code>ra</code>	return address / (pc,ib) link vector	caller

Table 4.3.: 16-bit register assignment

A. Appendix

A.1. Opcode summary — 16-bit

15	13	12	10	9	7	6	2	1	0	
operand						opcode		size		
uimm						00000		00		break uimm9
simm						00001		00		j simm9 × 2
simm						00010		00		b simm9 × 2
simm						00011		00		ibj simm9 × 64
fun	uimm					00100		00		link.i64 fun3, ib64(uimm6)
rc	uimm					00101		00		movh.i64 rc, ib32(uimm6)
rc	uimm					00110		00		movw.i64 rc, ib64(uimm6)
rc	simm					00111		00		movi.i64 rc, simm6
rc	simm					01000		00		addi.i64 rc, simm6
rc	uimm					01001		00		srli.i64 rc, uimm6
rc	uimm					01010		00		srai.i64 rc, uimm6
rc	uimm					01011		00		slli.i64 rc, uimm6
rc	uimm					01100		00		addh.i64 rc, ib32(uimm6)
rc	uimm					01101		00		leapc.i64 rc, ib32(uimm6)(pc)
rc	uimm					01110		00		loadpc.i64 rc, ib32(uimm6)(pc)
rc	uimm					01111		00		storepc.i64 rc, ib32(uimm6)(pc)
rc	rb	uimm				10000		00		load.i64 rc, (uimm3 × 8)(rb)
rc	rb	uimm				10001		00		store.i64 rc, (uimm3 × 8)(rb)
rc	rb	fun				10010		00		compare.i64 rc, rb, fun3
rc	rb	fun				10011		00		logic.i64 rc, rb, fun3
rc	rb	ra				10100		00		pin.i64 rc, rb, ra
rc	rb	ra				10101		00		and.i64 rc, rb, ra
rc	rb	ra				10110		00		or.i64 rc, rb, ra
rc	rb	ra				10111		00		xor.i64 rc, rb, ra
rc	rb	ra				11000		00		add.i64 rc, rb, ra
rc	rb	ra				11001		00		srl.i64 rc, rb, ra
rc	rb	ra				11010		00		sra.i64 rc, rb, ra
rc	rb	ra				11011		00		sll.i64 rc, rb, ra
rc	rb	ra				11100		00		sub.i64 rc, rb, ra
rc	rb	ra				11101		00		mul.i64 rc, rb, ra
rc	rb	ra				11110		00		div.i64 rc, rb, ra
uimm						11111		00		illegal uimm9

References

- [1] Howard H. Aiken and Grace Hopper. *A Manual of Operation for the Automatic Sequence Controlled Calculator*. Tech. rep. Describes the Harvard Mark I architecture with dedicated instruction memory. Harvard University Computation Laboratory, 1946.
- [2] AT&T Bell Laboratories. *UNIX[®] System V Release 4: Assembler and Machine-Level Debugging Guide*. Describes AT&T assembler directives and syntax used in UNIX[®] System V Release 4. Prentice Hall, 1990. ISBN: 0-13-947116-4.
- [3] J. Cocke and V. Markstein. “The Evolution of RISC Technology at IBM”. In: *IBM Journal of Research and Development* 34.1 (1990). Reviews the IBM 801’s architecture and its influence on later RISC systems, pp. 4–11. DOI: 10.1147/rd.341.0004.
- [4] DWARF Debugging Information Format Workgroup. *LEB128 (Little Endian Base 128) Encoding*. Section 7.6: Variable Length Data. Free Standards Group, 2006. URL: <https://dwarfstd.org/doc/Dwarf3.pdf>.
- [5] Free Software Foundation. *Using as: The GNU Assembler*. Part of GNU Binutils. GNU Project. 1991. URL: <https://sourceware.org/binutils/docs/as/>.
- [6] Advanced Micro Devices Inc. “Accessing an extended register set in an extended register mode”. Patent US6877084B1. Filed 2001-04-02. Granted 2005-04-05. Expired 2023-06-18. Apr. 2005. URL: <https://patents.google.com/patent/US6877084B1>.
- [7] Cray Research LLC. “Data processing system for processing one and two parcel instructions”. Patent US5717881A. Filed 1995-06-07. Granted 1998-02-10. Expired 2015-02-10. Feb. 1998. URL: <https://patents.google.com/patent/US5717881A>.
- [8] John von Neumann. *First Draft of a Report on the EDVAC*. Tech. rep. Seminal report introducing the stored-program concept, later known as the Von Neumann architecture. Moore School of Electrical Engineering, University of Pennsylvania, 1945.
- [9] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware Software Interface*. 5th. Describes the classic 5-stage RISC pipeline: fetch, decode, execute, memory, write-back. Morgan Kaufmann, 2013. ISBN: 9780124077263.
- [10] Ray J. Solomonoff. “A Formal Theory of Inductive Inference. Parts I and II”. In: *Information and Control* 7.1-2 (1964). Introduces the three-tape Universal Prefix Turing Machine model, pp. 1–22, 224–254. DOI: 10.1016/S0019-9958(64)90223-2.
- [11] Alan M. Turing. “On Computable Numbers, with an Application to the Entscheidungsproblem”. In: *Proceedings of the London Mathematical Society*. 2nd ser. 42 (1936). A theoretical model of computation, pp. 230–265. DOI: 10.1112/plms/s2-42.1.230.